

Security Risks of Third-Party APIs & Developer Behavior

Hien Long Nguyen
CISC 3600: *Secure Cyber Networks*
Fordham University
New York City, USA
Inguyen106@fordham.edu

Abstract—Modern applications increasingly rely on interconnected ecosystems of third-party APIs and development tools, which introduces significant security risks because these components are often trusted without caution. This paper examines the April 2026 Vercel security breach as a case study, analyzing how the compromise of a third-party OAuth application enabled attackers to gain unauthorized access to internal environments. The paper further explores how similar risks propagate through dependency ecosystems such as npm, using the 2026 Axios supply chain. In addition, this paper analyzes the growing problem of secret leakage in development workflows, drawing on GitGuardian’s findings on exposed secrets in public GitHub commits. Finally, the paper investigates the security implications of AI-assisted development through recent analysis of AI-generated code. The paper concludes by discussing mitigation strategies and developer behavior.

Keywords— software supply chain security, OAuth, API security, secret management, AI-assisted development, credential leakage

I. INTRODUCTION

The current trend in software development is that software are being built on layers of third-party APIs, using another layer of third-party tools. This has led to innovation in the software development space due to the convenience of integrating already-written code, but it has also introduced a class of security risks due to the blind trust that developers place on external tools. Recent incidents such as the security breach from Vercel have demonstrated that security compromise does not necessarily come from the codebase, but rather from the environment surrounding it. Furthermore, with the increasingly intertwined nature of codebases with third-party tools, a single compromise could ripple through the production system and cause large-scale damage.

II. THE VERCEL SECURITY BREACH

A. Background

The April 2026 Vercel security breach provides a useful case study for understanding how third-party integrations can cause unprecedented security vulnerabilities. Vercel, also known for the Next.js framework, is a popular cloud development platform used to deploy and host web applications. Because of its role and popularity, Vercel sits near sensitive parts of the software development cycle, including source repositories, environment variables, API tokens, and deployment settings.

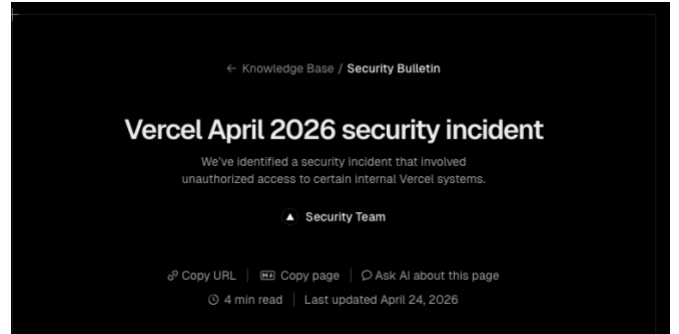


Fig. 1. Vercel April 2026 security incident bulletin interface [1]

B. Official Incident Report

According to Vercel’s official incident report, the issue started with Context.ai, a third-party AI tool that had its Google Workspace OAuth application compromised [1]. Vercel stated that the OAuth compromise potentially affected “hundreds of users across many organizations,” meaning the issue was not isolated to Vercel alone.

Within Vercel’s environment, the attacker gained access to a specific employee account, which provided entry into a limited portion of the internal system. Vercel reported that the attacker was able to access and enumerate “non-sensitive” environment variables associated with a subset of customer projects.

In response, the company took immediate steps to contain the breach, including revoking unauthorized access, conducting audits of affected systems, and identifying accounts that show suspicious activity. Vercel has also published a guidance to impacted users, recommending that they review logs and rotate keys and credentials.

Despite the mitigation efforts, the report acknowledges limitations in determining the full scope and damage of the breach. Thus, it is necessary to further analyze how the attack unfolded.

C. Attack Chain Analysis

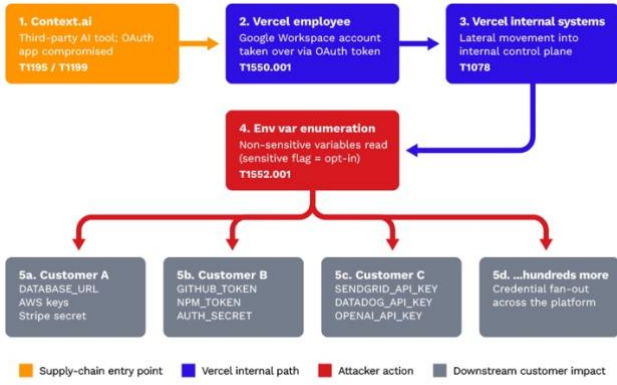


Fig. 2. Vercel breach attack chain. Source: Peter Girmus, “The Vercel Breach: OAuth Supply Chain Attack Exposes the Hidden Risk in Platform Environment Variables” [2]

The Vercel breach can be understood as an attack chain that exploited trust relationships.

In the first stage, attackers compromised Context.ai. This compromise has been linked to credential theft from a Context.ai employee, which enabled attackers to access Context.ai’s infrastructure and exfiltrate OAuth tokens associated with its users. These tokens functioned as delegated credentials and did not require the user’s password, effectively bypassing traditional authentication [2].

In the second stage, the attacker used the compromised OAuth tokens to access a Vercel employee’s Google Workspace account. Because OAuth tokens inherit the permissions granted to the application, the attacker gained access to resources such as email, drive, and calendar access.

The third stage involved lateral movement within Vercel’s internal systems. Modern enterprise environments commonly rely on single sign-on (SSO), where access to an identity provider like Google Workspace can grant access to multiple connected platforms. In this case, the compromised Workspace account gave the attacker access to the employee’s Vercel account which gave them access to Vercel’s internal systems.

Once inside the environment, the attacker enumerated available resources and gained access to customer projects’ environment variables.

D. Damage and Impact

External reports state that threat actors attempted to monetize the breach by offering allegedly stolen data on BreachForums for approximately \$2 million [3]. Although the claims remain unverified, it gives insight into the perceived value of compromised data. Furthermore, the incident introduces significant operational consequences for both Vercel and its users. Affected organizations and individuals are to assume credential exposure and are advised to take immediate measures, which are costly in terms of time, engineering effort, and could create service disruption. Beyond immediate technical exposure, the breach introduces longer-term risks, as incidents like these can reduce user trust in the platform.

The same type of trust problems also exists in npm ecosystems, which the next section will explore.

III. NPM & THE AXIOS SUPPLY CHAIN COMPROMISE

A. Background

Most software systems rely on dependency ecosystems such as npm, as applications are often built from large collections of third-party packages.

B. Npm Analysis

Decan et al. conducted a study of the npm dependency network consisting of over 610,000 JavaScript packages, more than 4.2 million package releases, and over 20 million runtime dependencies [4].

The study identified over 133,000 packages that depended on vulnerable packages, with more than 72,000 packages containing at least one release affected by an upstream vulnerability. These affected dependencies accounted for over 411,000 vulnerable releases.

The paper also found that vulnerabilities often remain active for long periods of time. Using survival analysis, the authors discovered that it takes more than 24 months to discover 50% of all vulnerabilities, while high-severity vulnerabilities required approximately 31 months before discovery.

These findings show that while npm is widely trusted and used, there exists vulnerable packages that could cause major risks if unidentified.

C. The Axios Compromise

The 2026 Axios npm supply chain compromise provides an example of how these risks are exploited in practice. Axios, a widely used JavaScript HTTP client, was targeted when attackers published malicious versions (1.14.1 and 0.30.4) after compromising the package distribution process [5]. Instead of modifying Axios’s core functionality, the attackers introduced a malicious dependency that executed automatically during installation through npm’s scripts.

This attack leverages a key feature of npm: when a package is installed, npm recursively installs all required dependencies and executes associated scripts without requiring explicit user approval. In the Axios incident, this behavior allowed the malicious dependency to run immediately upon installation, establishing a connection to the attacker and retrieving additional payloads [5].

The impact of such attacks is amplified by versioning and update mechanisms. Many projects specify dependency ranges (e.g., ^1.14.0), allowing package managers to automatically install newer compatible versions. During the exposure window, any system resolving Axios dependencies could unknowingly upgrade to the malicious versions [5].

Without direct oversight and research, developers can unknowingly install these malicious packages onto their projects.

IV. SECRET LEAKAGE AND DEVELOPER HABITS

A. Hardcoded Secrets

Developers often neglect the process of secret management. Analysis from the GitGuardian’s *State of Secrets Sprawl* 2026 report implies that many exposed secrets originate from common development practices as developers frequently hardcode credentials during testing,

debugging, or initial development, with the intention of removing them later. However, these credentials are often accidentally committed to Git and published online, which becomes a permanent vulnerability if the secrets are not rotated.

According to report, approximately 28.65 million new hardcoded secrets were detected in public GitHub commits in 2025, which was a 34% increase from the previous year, and was the largest single-year increase recorded [6].

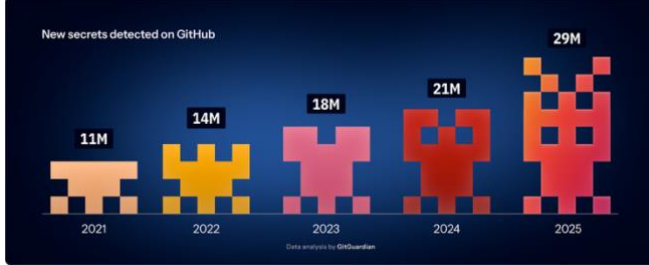


Fig. 3. Number of new secrets detected on GitHub from 2021–2025. Source: GitGuardian, “State of Secrets Sprawl 2026.” [6]

B. Rotating Exposed Secrets

Even after exposure, many of these secrets are not rotated and remain active for extended periods of time which is consistent with the findings by Decan et al. As shown in Fig. 4, 77% of exposed keys from 2025 remain valid through January 2026.

One potential reason behind this issue is that secret rotation is often a disruptive process in development. API keys and access tokens are embedded across multiple services, including backend systems, cloud platforms, and CI/CD pipelines. Rotating a single key often means updating configurations across several interconnected components, which greatly slows down development. As a result, developers may delay rotating secrets, especially if the impact is not immediately clear.

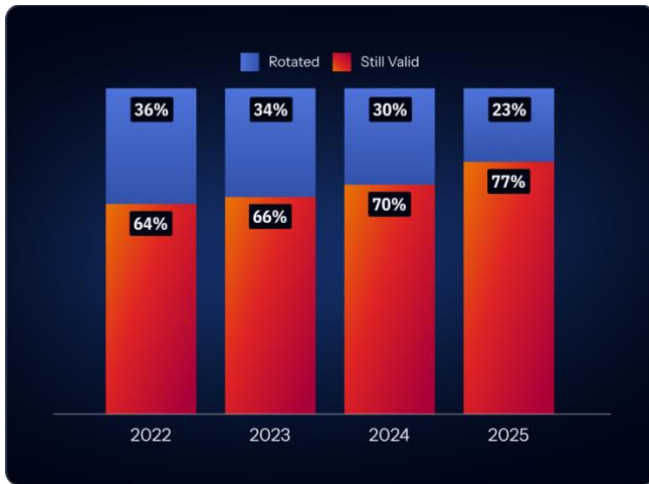


Fig. 4. Percentage of exposed secrets that remain valid versus rotated (2022–2025). Source: GitGuardian, “State of Secrets Sprawl 2026.”

C. Public vs. Internal Secret Leakage

The report found 24,008 unique secrets across GitHub, which makes 5.6% of all public repositories contain at least one secret. As shown in Fig. 5, the issue is more prominent

in internal repositories, with 32.2% containing at least one secret.

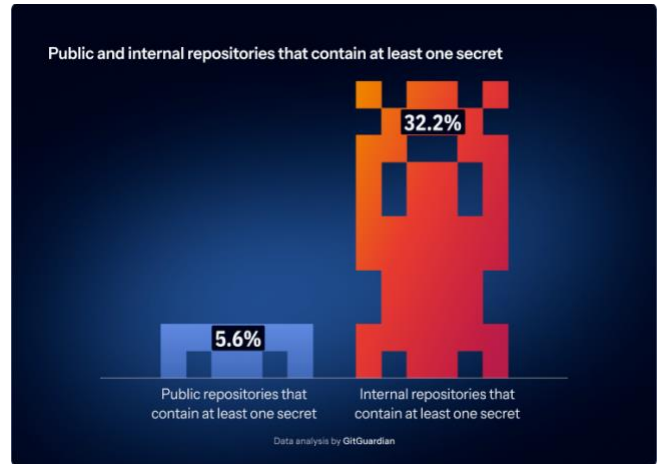


Fig. 5. Percentage of public and internal repositories containing at least one secret. Source: GitGuardian, “State of Secrets Sprawl 2026.”

This disparity suggests that developers often have a false sense of security when working with internal repositories. Because internal systems are perceived as less exposed, developers may be more casual with secrets.

V. AI-ASSISTED DEVELOPMENT AND EMERGING RISKS

A. Background

Developer negligence is further amplified by the rapid adoption of AI tools. While these tools aid developers in production, developers often overlook security in favor of speed. A recent study analyzing AI-generated code across public GitHub repositories provides useful evidence for this issue [7].

B. Methodology

The study examined 7,703 files explicitly attributed to major AI tools such as ChatGPT, GitHub Copilot, and Amazon CodeWhisperer using CodeQL static analysis [7]. To evaluate security issues, the study relied on the Common Weakness Enumeration (CWE) framework, which is a standardized classification system for software vulnerabilities. CWE categories represent known types of weaknesses, such as improper input validation, insecure authentication, or exposure of sensitive data.

C. Findings

Using these, the researchers identified 4,241 instances of CWE-classified weaknesses across 77 distinct vulnerability types. Although 87.9% of the analyzed AI-generated code did not contain detectable vulnerabilities, the remaining portion revealed consistent patterns of security risk. Notably, vulnerability rates varied significantly across programming languages, with Python having higher vulnerability rates (16.18%-18.50%) compared to JavaScript (8.66%-8.99%) and TypeScript (2.50%-7.14%) [7]. This is attributed to both the training data and structural characteristics of the languages. Nevertheless, this shows that relying entirely on AI for code generation is problematic. The study also highlights a shift in development practices, as AI tools are

frequently used for producing documentation and boilerplate components in 39% of the collected files.

D. Implications

This trend reinforces the pattern observed in the Vercel breach and in the GitGuardian report. As development speeds up, security does not scale at the same rate, leading to an increased likelihood of vulnerabilities in production systems. As shown in Fig. 6, the number of AI-assisted commits has increased rapidly over time, growing from approximately 1.3K in October 2024 to over 3.5 million by late 2025 [6]. This increase also reflects the scale of which AI-tools are being integrated into development workflows.

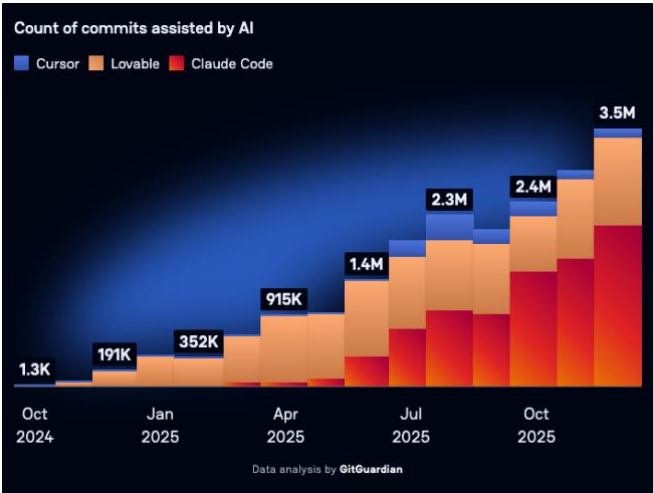


Fig. 6. Count of commits assisted by AI Oct 2024 – Oct 2025. Source: GitGuardian, “State of Secrets Sprawl 2026.”

The report further shows that the types of secrets being exposed are changing. As shown in Fig. 7, credentials for AI services are being created faster than other categories of secrets, with OpenRouter increasing by 48x, DeepSeek by 23x, Brave Search by 13x, and Firecrawl by 9x [6]. GitGuardian also notes that these leaks are more likely to bypass controls designed around traditional developer workflows, suggesting that AI integration is expanding the secret-management problem.



Fig. 7. Growing Specific AI-related Secret Types. Source: GitGuardian, “State of Secrets Sprawl 2026.”

VI. MITIGATION STRATEGIES AND FUTURE IMPLICATIONS

A. Core Security Practices

A fundamental mitigation strategy is enforcing the principle of least privilege (PoLP), where systems and applications are granted only the minimum permissions required to function [8]. In OAuth-based systems, limiting token scopes and conducting regular access audits can significantly reduce the attack surface. In addition, adopting zero trust architecture (ZTA) ensures continuous verification of identity and access, even within internal environments [9]. Applying these to third-party integrations and development tools can help prevent lateral movement after an initial compromise, as observed in the Vercel attack chain.

B. AI Mitigation & Future Implications

For AI-assisted development, maintaining human oversight is critical. AI-generated code should be treated as untrusted and subjected to the same review standards as externally sourced code. This includes static analysis, dependency auditing, credential scanning, and manual review of permissions. Developers should also be trained to critically evaluate generated implementations instead of assuming that functional code is inherently secure.

VII. CONCLUSION

The Vercel breach demonstrates that modern cybersecurity risks no longer originate only from application code, but from the ecosystem of third-party services and the development workflows surrounding it. The attack showed how trust relationships between external tools can be exploited to gain access to internal information without directly compromising the application itself. Similar patterns are visible in dependency ecosystems such as npm, where the Axios compromise demonstrated how malicious packages can propagate through trusted installation processes. These risks are largely due to developer behavior. The widespread exposure of hardcoded credentials, combined with the failure to rotate compromised secrets, reveals how convenience and development speed often take priority over security.

Taken together, these issues reflect a broader shift in cybersecurity. Systems are becoming increasingly interconnected and dependent on external components. As a result, vulnerabilities are no longer isolated, and developers must be cautious of these risks.

REFERENCES

- [1] Vercel, “Vercel April 2026 Security Incident – Who is impacted,” 2026. [Online]. Available: <https://vercel.com/kb/bulletin/vercel-april-2026-security-incident#who-is-impacted>
- [2] P. Girnus, “The Vercel Breach: OAuth Supply Chain Attack Exposes the Hidden Risk in Platform Environment Variables,” 2026. [Online]. Available: https://www.trendmicro.com/en_us/research/26/d/vercel-breach-oauth-supply-chain.html
- [3] M. Siman Tov Bustan and N. Zadok, “Supply Chain Attack Hits Vercel: User Data is Being Sold on BreachForums For \$2M,” OX Security, 2026. [Online]. Available: <https://www.ox.security/blog/vercel-context-ai-supply-chain-attack-breachforums>
- [4] A. Decan, T. Mens, and E. Constantinou, “On the impact of security vulnerabilities in the npm package dependency network,” in Proc. 15th Int. Conf. Mining Software Repositories (MSR), 2018. [Online]. Available: <https://dl.acm.org/doi/10.1145/3196398.3196401>
- [5] Microsoft Threat Intelligence and Microsoft Defender Security Research Team, “Mitigating the Axios npm supply chain compromise,” Microsoft Security Blog, Apr. 1, 2026. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise/>
- [6] GitGuardian, “The State of Secrets Sprawl Report 2026,” 2026. [Online]. Available: <https://www.gitguardian.com/state-of-secrets-sprawl-report-2026>
- [7] M. Schreiber and P. Tippe, “Security vulnerabilities in AI-generated code: A large-scale analysis of public GitHub repositories,” arXiv preprint arXiv:2510.26103, Oct. 2025. [Online]. Available: <https://arxiv.org/abs/2510.26103>
- [8] NIST, “Security and Privacy Controls for Information Systems and Organizations,” NIST SP 800-53 Rev. 5, 2020.
- [9] NIST, “Zero Trust Architecture,” NIST SP 800-207, 2020.